Operations Research

Publication details, including instructions for authors and subscription information:
http://pubsonline.informs.org

Selection and Ordering Policies for Hiring Pipelines via Linear Programming

Boris Epstein, Will Ma

https://pubsonline.informs.org/journal/opre

OPERATIONS RESEARCH
Vol. 72, No. 5, September–October 2024, pp. 2000–2013
ISSN 0030-364X (print), ISSN 1526-5463 (online)

Crosscutting Areas

Selection and Ordering Policies for Hiring Pipelines via Linear Programming

Boris Epstein,[a,*] Will Ma[a]

[a] Graduate School of Business, Columbia University, New York, New York 10027

*Corresponding author

Contact: bepstein25@gsb.columbia.edu, https://orcid.org/0000-0002-8988-381X (BE); wm2428@gsb.columbia.edu, https://orcid.org/0000-0002-2420-4468 (WM)

Received: February 3, 2023
Revised: October 13, 2023
Accepted: January 9, 2024
Published Online in Articles in Advance: February 28, 2024

Area of Review: Markets, Platforms, and Revenue Management

https://doi.org/10.1287/opre.2023.0061

Abstract. Motivated by hiring pipelines, we study three selection and ordering problems in which applicants for a finite set of positions are interviewed or sent offers. There is a finite time budget for interviewing/sending offers, and every interview/offer is followed by a stochastic realization of discovering the applicant's quality or acceptance decision, leading to computationally challenging problems. In the first problem, we study sequential interviewing and show that a computationally tractable, nonadaptive policy that must make offers immediately after interviewing is near optimal, assuming offers are always accepted. We further show how to use this policy as a subroutine for obtaining a polynomial-time approximation scheme. In the second problem, we assume that applicants have already been interviewed but only accept offers with some probability; we develop a computationally tractable policy that makes offers for the different positions in parallel, which can be used even if positions are heterogeneous, and is near optimal relative to a policy that can make the same total number of offers one by one. In the third problem, we introduce a parsimonious model of overbooking where all offers are sent simultaneously, and a linear penalty is incurred for each acceptance beyond the number of positions; we provide nearly tight bounds on the performance of practically motivated value-ordered policies. All in all, our paper takes a unified approach to three different hiring problems based on linear programming. Our results in the first two problems generalize and improve the existing guarantees in the literature that were between $1/8$ and $1/2$ to new guarantees that are at least $1 - 1/e \approx 63.2\%$. We also numerically compare three different settings of making offers to candidates (sequentially, in parallel, or simultaneously), providing insight into when a firm should favor each one.

Supplemental Material: The online appendices are available at https://doi.org/10.1287/opre.2023.0061.

Keywords: hiring • stochastic probing • approximation algorithm • adaptivity gap

1. Introduction

Hiring the right personnel is one of the most important factors in the success of an enterprise. That being said, carrying out an efficient and timely recruitment process can be challenging in practice. Several difficulties arise when hiring, such as (but not limited to) deciding when to carry out the process, dealing with imperfect information, and making operational decisions at the time that the process is being carried out. This last aspect of the recruitment process raises several questions that can be studied through an algorithmic lens.

A typical recruitment process starts with a firm making a call for applications. An application usually consists of a resume and potential complementary materials. Based on this (imperfect) information, the firm decides *who* is going to be interviewed and *in which order* are the interviews going to be conducted. Both these aspects are relevant because a recruitment process cannot go on forever. That is, there is not enough time to interview every single applicant, and the firm can choose who to interview next depending on the outcomes of past interviews.

Applicants become candidates once they are interviewed. After all interviews are conducted, the firm sends offers to the candidates it wishes to hire, given a limited set of positions. However, there are many possible ways to send offers to candidates. The first natural approach would be to *sequentially* send offers to candidates. By this we mean: send an offer, wait for the response of the candidate, and (if there is still a position available) carry on with the next offer. As in the interviewing process, the *order* in which the offers are sent becomes a relevant decision. A second approach, applicable only to a firm hiring more than one person, is to save time by sending offers in *parallel*. This means, for each position remaining to fill: send an offer, wait for the response of the candidate, and (if still unfilled) carry on with subsequent offers. The final approach, which may be desirable under a tight timeline or to avoid revealing preferences among the candidates sent offers, is to send all offers *simultaneously*. However, it runs the risk of hiring more people than positions available, which comes at a cost.

To answer these operational questions and compare the different modes of sending offers, we study three different models of hiring processes that build off existing work.

1.1. First Model: Sequential Interviewing, a.k.a. ProbeTop-*k*

In the first model, we assume that the firm has to hire up to k people from a pool of n applicants. Each applicant has a random value unknown to the firm carrying out the hiring process. The firm has access to distributional knowledge of these values coming from the applicants' resumes and complementary material submitted. The firm can interview applicants to find out the realization of their values, but there is a limit of T on the number of interviews conducted. The realization of the value of the applicant becomes known to the firm immediately after carrying out the interview. This realization should be interpreted as the applicant's *expected* value to the firm given the interview (and conditional on them accepting the offer). We treat this value as deterministic, which does not lose generality for a risk-neutral firm that maximizes its expectation. We further assume that realizations from interviews are independent across applicants. After all interviews are carried out, the firm can choose the best k interviewed candidates to be hired, who are assumed to accept their offers with probability 1. The goal of the firm is to maximize the expected sum of values of the hired personnel. This problem is exactly the ProbeTop-k problem, as described in Fu et al. (2018).

In this problem, we can further distinguish between different classes of policies. First, we distinguish between *adaptive* and *nonadaptive* policies. Adaptive policies can decide the order of the interviews on the fly, choosing who to interview next depending on the outcomes of previous interviews. Nonadaptive policies, in contrast, have to fix an interview order before the process starts, which, although restrictive, is attractive from an ease-of-implementation perspective. We also distinguish between *committed* and *noncommitted* policies. Committed policies have to irrevocably decide whether to hire each candidate immediately after interviewing them and discovering their value. Noncommitted policies, in contrast, can carry out all interviews and choose the k highest realized values in hindsight. Using committed policies could be attractive from a practical point of view, as waiting until the end incurs the risk that candidates accept offers from competing firms in the meantime. We are interested in bounds on how costly it is to restrict the firm to use policies that are nonadaptive and committed. The former is quantified in the literature by the widely studied notion of *adaptivity gap*: the worst-case ratio between the performance of general policies versus algorithms that are restricted to be nonadaptive. Our results will bound the "adaptivity-commitment gap," in which the algorithm is restricted to be both nonadaptive and committed.

1.1.1. Relation to Free-Order Prophets. The Free-Order Prophet Inequality problem is the special case of ProbeTop-k where $T = n$ and only committed policies are allowed. (When $T = n$, the constraint of T interviews is not binding, and hence noncommitted policies can just trivially interview all applicants.) Typically, in prophet inequalities, the benchmark can see all applicants' values in advance and simply choose to interview and hire the k overall highest-valued candidates. Such a benchmark is *too powerful to compare against* in our more general problem if n is much larger than T because the benchmark sees all n realizations, whereas the algorithm can only interview T applicants. This is why in our general ProbeTop-k problem, we compare to an optimal (adaptive, noncommitted) algorithm that is still bound by T interviews that must be decided without any prophetic information, making our comparison different from prophet inequalities.

1.1.2. Relation to Sequential Offering. In the Sequential Offering model of Purohit et al. (2019), applicants are assumed to have already been interviewed but have uncertainty about whether they accept an offer. The firm knows, for each candidate, how likely it is that they will accept an offer, and the value they add to the firm, should they accept. The firm has time to send at most T offers and wants to maximize the expected total value of up to k candidates who accept their offers. This setting is closely related to the special case of ProbeTop-k with weighted Bernoulli distributions, where the values take a positive realization with some probability (representing an acceptance) or zero with the remaining probability (representing a rejection). The subtle difference is that an accepted offer cannot be withdrawn by the firm in the Sequential Offering model, whereas in the ProbeTop-k model, the firm can turn down a candidate even if their value turns out to be positive. We will show that our algorithm satisfies properties that makes it admissible for this Sequential Offering problem too and improve on the results of Purohit et al. (2019).

1.2. Second Model: Parallel Offering

We also study a Parallel Offering model, in which a firm again has to hire people to fill k positions. However, we now allow for heterogeneous positions, where a candidate may have different potential values for different job positions. We assume that all interviews have already been conducted, leaving us with a pool of n desirable candidates. After conducting all interviews the firm learned, for each candidate, how valuable they are for each of the available positions and how likely it is for each candidate to accept an offer for each of the available positions. The firm must now decide how to send offers in T parallel offering rounds. At each round, the firm can send an offer for each of the positions that have not yet been filled by a candidate. When a candidate receives an offer, they can either accept or reject it, with the assumption that they cannot receive an offer for another position if they rejects it (and hence candidates do not try to anticipate offers they might receive later). The goal of the firm is to maximize the expected sum of values of hired candidates. We develop a nonadaptive algorithm that can be computed efficiently and performs competitively in comparison to adaptive and even relaxed sequential algorithms.

This model generalizes a parallel offering model also introduced in Purohit et al. (2019), which is similar but has k identical instead of heterogeneous positions. Our Parallel Offering model is not only more general, but we also derive stronger performance guarantees.

1.3. Third Model: Simultaneous Offering

Finally, we study the Simultaneous Offering model, again for a firm hiring to fill k positions. As in the Parallel Offering model, the firm has already conducted all interviews, resulting in a pool of n candidates for which it knows how valuable each candidate is to the firm and how likely each candidate is to accept an offer. The firm must decide on a subset of candidates who will receive an offer (all at the same time). This subset can be of any size, so a possible outcome is that more than k candidates accept an offer. If that is the case, the firm pays a penalty for each candidate who accepted an offer beyond the capacity k. This penalty can be thought of as the cost of withdrawing an offer, or the cost of creating a new position in the firm. We analyze the performance of value-ordered policies, which send offers to candidates above a value threshold (regardless of their probability of acceptance). We derive near-optimal approximation guarantees for these policies.

Table 1 contains a comparison of all the models studied/captured in this paper.

1.4. Outline of Results

We will generally say that our algorithm is α-approximate if its expected total value collected is at least α times that of an optimal algorithm, from a larger class. We call $\alpha \in [0, 1]$ the approximation factor. Typically, this terminology is used when comparing to the optimal algorithm from the *same* class. Our results imply lower bounds on the approximation factor α under the typical terminology because we are comparing against a larger class. All our results assume that distributions have *finite support* and are explicitly input in the form of (value, probability) pairs with binary encoding. All of our algorithms are polynomial-time under this form of input.

For the problem of computing the optimal algorithm within a fixed class, no computational hardness results are known for any of the problems we study (ProbeTop-k, Parallel Offering, Simultaneous Offering), to our knowledge. Nonetheless, our algorithms still have some of the currently best-known approximation factors, which also hold when comparing to a larger class.

1.4.1. Sequential Interviewing a.k.a. ProbeTop-*k* Problem. We develop a polynomial-time algorithm that is nonadaptive, committed, and achieves a $(1-e^{-k}k^k/k!)$ approximation factor relative to an optimal adaptive, noncommitted algorithm (Theorem 1, Section 4.4). The approximation factor of $(1-e^{-k}k^k/k!)$ equals $1-O(1/\sqrt{k})$ by Stirling's approximation and is always at least $1-1/e \approx 0.632$ (when $k=1$). It is tight relative to the linear program (LP) benchmark we compare against (Online Appendix B6).

Our results for this problem, although simple and clean, have broad implications. First, we can combine our algorithm with the work of Fu et al. (2018) to obtain a polynomial-time approximation scheme (PTAS) for ProbeTop-k (Corollary 1, Section 4.5). There exist PTAS's for ProbeTop-k restricted to nonadaptive algorithms (Segev and Singla 2021) and ProbeTop-k restricted to committed algorithms (Fu et al. 2018), but a PTAS for the general (adaptive, non-committed) ProbeTop-k problem appears unknown, assuming that k is part of the input. Second, the best-known existing guarantee on the adaptivity gap for ProbeTop-k was $1/2$ from Bradac et al. (2019) via an algorithm that is not necessarily polynomial time. We improve the lower bound on the adaptivity gap for ProbeTop-k from $1/2$ to $1-1/e$ using a nonadaptive, polynomial-time algorithm, and moreover show asymptotic optimality when the number of positions grows to infinity. Third, our $(1-e^{-k}k^k/k!)$-approximation algorithm can be directly applied to the Sequential Offering problem without loss (Corollary 2, Section 4.6), improving the previous-best approximation factor and adaptivity gap of $1/2$ (Purohit et al. 2019). As with Bradac et al. (2019), the adaptivity gap shown in Purohit et al. (2019) was achieved through a nonconstructive approach. Finally, our abstract treatment leads to an algorithm that works in the Free-Order Prophets setting (because it is committed); hence, our results extend to a free-order "prophet inequality" (comparing against a necessarily weaker benchmark) that allows an additional constraint of T on the number of interviews.

1.4.2. Parallel Offering Problem. For the Parallel Offering model, we develop an algorithm that is nonadaptive and achieves a $(1-1/e)$ approximation factor relative to

Table 1. Comparison Between Models

Model	Action performed	Result of action	Actions per time step	Moment of hiring	Bound on total hires
ProbeTop-k	Interview	Observe value of candidate	One	After last interview	Hard
Free-Order Prophets	Interview	Observe value of candidate	One	After each interview (irrevocable)	Hard
Sequential Offering	Send offer	Observe accept/reject decision	One	Upon acceptance of offer	Hard
Parallel Offering	Send offer(s)	Observe accept/reject decision(s)	One per position remaining	Upon acceptance of offer	Hard
Simultaneous Offering	Send offer(s)	Observe accept/reject decision(s)	n	Upon acceptance of offer	Soft (linear penalty)

an optimal (adaptive) algorithm (Theorem 2, Section 5.3). This result also both improves and generalizes existing results, in this case, the 1/8-approximate algorithm of Purohit et al. (2019) that works in the special case where all positions are identical. It is also tight relative to the LP benchmark, we compare against (Online Appendix C5). We further show that our algorithm obtains at least $(1-1/e)$ times what an optimal algorithm would obtain in the Sequential Offering problem with kT time steps, establishing a lower bound on the value of batching offers (Corollary 3, Section 5.4).

1.4.3. Simultaneous Offering Problem. We analyze the performance of *value-ordered* policies, which send offers to all candidates whose value is above a threshold. The idea of having a value threshold is practically motivated and the optimal value-ordered policy can be computed efficiently. We show that with no assumptions on the values of the candidates, value-ordered policies can have arbitrarily poor performance (Example 1, Section 6.1). Consequently, we assume that the valuations of candidates are lower bounded by a parameter $\tau \in (0,1)$ and provide a value-ordered policy that achieves at least a factor α_k^τ of what an optimal policy could achieve, where α_k^τ is an increasing function of k and τ (Theorem 3, Section 6.4) that satisfies $\lim_{k\to\infty}\alpha_k^\tau = 1$ for every $\tau \in (0,1)$ (Corollary 4, Section 6.4). We also provide an instance where no value-ordered policy can achieve a factor higher than β_k^τ of what an optimal policy could achieve, where β_k^τ is another increasing function of k and τ (Theorem 4, Section 6.4). We further characterize the region where $\alpha_k^\tau = \beta_k^\tau$ (Proposition EC.6, Section D.8). In particular, our bound is tight for all k if $\tau \leq 1/2$.

1.4.4. Numerical Study. In Online Appendix E, we numerically simulate the three different models of sending offers (sequential, parallel, simultaneous) using the same candidate pool generation process as Purohit et al. (2019). We illustrate the improved performances offered by our new policies and compare the performances attainable across the different models, providing insight into when the firm should favor sending offers sequentially, in parallel, or simultaneously. For instance, there is surprisingly little value to gain from an algorithm that sends offers in parallel, as opposed to sending single offers sequentially and being highly adaptive to the number of remaining positions, unless the horizon for making offers is extremely short. For Simultaneous Offering, we demonstrate that value-ordered policies are generally desirable unless there is both a small number of initial positions and a high cost of overage. In that case, it is better to identify the highest-valued "safe" candidates (with a high probability of acceptance) to reduce the variance in the number of accepted offers. We acknowledge that these insights are based on the specific generative model of Purohit et al. (2019), and they do not necessarily hold in general.

2. Related Work

2.1. Stochastic Probing and Matching

Our work is closely related to the general stochastic probing problem studied in Gupta and Nagarajan (2013) and Gupta et al. (2016). These papers study the problem of sequentially probing elements to maximize the sum of the weights of a selected subset. In their setting they consider more general sets of "outer" constraints to be satisfied by the probed elements and "inner" constraints to be satisfied by the selected elements. In this language, ProbeTop-k considers the outer constraint to be the T-uniform matroid and the inner constraint to be the k-uniform matroid. Their work is different from ours in that it only allows the probe to have two outcomes (active or inactive) and active probes must be irrevocably included in the final subset. Bradac et al. (2019) introduce the multiple-type general stochastic probing problem. In this work, they only work with outer constraints and include the inner constraints by allowing submodular functions instead of modular functions.

In Bradac et al. (2019) they show that the adaptivity gap is exactly 1/2 for the stochastic probing problem with monotone submodular functions under prefix-closed probing constraints. Their proof is not constructive in the sense that the algorithm for their lower bound requires the optimal decision tree as an input. The best-known nonadaptive algorithm for which is known that it can be computed in polynomial time is due by Gupta et al. (2017), which achieves a $1/3$ approximation for submodular and max-of-sum-of-singletons (XOS) functions under prefix-closed probing constraints.

2.2. PTAS-Type Results

Fu et al. (2018) develop PTASs for a class of dynamic programs that includes ProbeTop-1 over general (adaptive, noncommitted) algorithms and ProbeTop-k over committed algorithms. They also provide a $(1-\varepsilon)-$approximation for ProbeTop-k over adaptive, noncommitted algorithms whose runtime is polynomial in $1/\varepsilon$ but exponential in k. Segev and Singla (2021) develop the improved notion of efficient polynomial-time approximation schemes (EPTASs) for several related problems, including ProbeTop-k over nonadaptive algorithms. Their EPTAS works differently when k is small or large. We use the same LP relaxation as theirs for the large k case, although our rounding scheme and analysis are very different. None of these PTAS-type results use nonadaptive, committed policies to approximate the best adaptive, noncommitted policies, as in our paper.

2.3. Prophet Inequalities

Our work has the flavor of Prophet Inequalities in that we are deciding online whether to accept values drawn from known distributions and comparing against a supernatural benchmark. Classical works in Prophet Inequalities (Krengel and Sucheston 1977, 1978; Samuel-Cahn 1984; Kleinberg and Weinberg 2012; Esfandiari et al. 2017) assume that the order is adversarial or random, whereas we are studying the *free-order* variant where the order can be decided (Hill 1983, Agrawal et al. 2020, Beyhaghi et al. 2021). However, because of the constraint of T time steps, as discussed earlier, it is important to emphasize that the benchmark we are comparing against is also bound by T time steps, so we are not comparing against (and it is impossible to compare against) the true prophet. For sequential interviewing, the approximation factor we derive of $(1-e^{-k}k^k/k!)$ has also been established for free-order by Yan (2011) and extended in Arnosti and Ma (2023) to random-order but neither of these works allows for a constraint of T time steps.

2.4. Simultaneous Offering Model

Our Simultaneous Offering model and in particular the linear penalty cost is motivated by static overbooking models in revenue management (Gallego and Topaloglu 2019, chapter 3). However, our decision is different in that we are selecting a *subset* of candidates, whereas they are setting a single booking limit (possibly one for each fare class). The simple family of heuristics for which we provide an approximation guarantee also has no analog in overbooking. Our problem also shares many aspects with the one studied in Cominetti et al. (2010), where a subset of customers are sent last-minute offers for a limited number of items. Their model differs from ours in that if more items than available inventory are sold, they only collect value from a random subset of size k, whereas our model collects the value of all acceptances but has to pay a linear penalty.

2.5. Concurrent Work

Parallel to our work, Gallego and Segev (2022) have studied and obtained results for the ProbeTop-k problem. Although both works aim to derive nonadaptive algorithms for the problem, theirs is different from ours in several aspects. First, their models assume that the valuations of the candidates come from independent random variables with either general distributions or continuous distributions, whereas our work focuses on random variables with finite support. Although the bound they obtain for continuous random variables matches the $1-e^{-k}k^k/k!$ bound we obtain for finitely-supported random variables, the bound they obtain for general random variables is $1/2$. Another aspect that distinguishes our work from Gallego and Segev (2022) is the relaxations used to benchmark the performance of algorithms. We use standard LP relaxations, whereas they introduce a novel benchmark consisting of a simple minimax problem.

3. Problem Formulations and Preliminaries

In this section, we formally state the problems studied in the paper. We first state the ProbeTop-k problem in Section 3.1. We then state the Parallel Offering problem in Section 3.2. We close by stating the Simultaneous Offering problem in Section 3.3.

3.1. ProbeTop-k Problem

In the ProbeTop-k ($\mathtt{ptk}$) problem, a firm faces the challenge of filling k positions out of a pool of n applicants. Each applicant i has a random, nonnegative value $V_i \sim F_i$, and F_i is known to the firm. Before the firm hires an applicant, an interview must be conducted. When the firm interviews applicant i, the realization of V_i becomes known to the firm. The firm can conduct at most T interviews in total and then can choose any k interviewed candidates to be hired. The goal of the firm is to maximize the sum of the values of the hired candidates. An instance of the problem is characterized by a tuple $I=(k,T,n,F)$, where $1 \le k \le T \le n$, and $F=\{F_i\}_{i\in[n]}$ is a collection of probability distribution functions. In this work, we focus on distributions supported on a finite set of non-negative values $\{r_j\}_{j\in[J]}$ and we use q_{ij} to denote $\mathbb{P}(V_i=r_j)$. We use $\mathcal{I}_{\mathtt{ptk}}$ to denote the set of all possible instances for the ProbeTop-k problem. We further use $\mathcal{I}^k_{\mathtt{ptk}} \subseteq \mathcal{I}_{\mathtt{ptk}}$ to denote the subset of instances where the number of positions is k.

For this problem, we define a policy as a function π that maps the remaining budget of interviews, the set of applicants that have not yet been interviewed and the realization of the values of the candidates that have already been interviewed to a decision of which applicant to interview next. Let $\Pi^{\mathtt{ptk}}$ be the set of all policies for ptk. For a policy $\pi \in \Pi^{\mathtt{ptk}}$ and an instance $I \in \mathcal{I}_{\mathtt{ptk}}$, let $R_\pi(I)$ be the expected reward of using policy π on the instance in question. For an instance $I \in \mathcal{I}_{\mathtt{ptk}}$ define $\mathsf{OPT}_{\mathtt{ptk}}(I) := \sup_{\pi\in\Pi^{\mathtt{ptk}}} R_\pi(I)$ as the expected reward of using the best possible policy on instance I. We call a policy $\pi \in \Pi^{\mathtt{ptk}}$ an α-approximation if

$$\inf_{I\in\mathcal{I}_{\mathtt{ptk}}} \frac{R_\pi(I)}{\mathsf{OPT}_{\mathtt{ptk}}(I)} \geq \alpha.$$

We distinguish between *adaptive* and *nonadaptive policies*. A nonadaptive policy has to decide an order in which to conduct the interviews before the process starts. An adaptive policy, in contrast, conducts the interviews sequentially and can use the outcomes of previous interviews to decide which candidate to interview next. We also distinguish between *committed* and *noncommitted* policies. After each interview, committed policies must irrevocably decide whether to hire the candidate or not. Noncommitted policies, in contrast, can interview T applicants and then choose the k highest realizations among them. Among other questions, we are interested in how well can the firm perform when restricted to using nonadaptive policies, committed policies, or both. To quantify this, we are interested in how good an approximation factor can be achieved by restricting the firm to using policies that are both nonadaptive and committed.

3.2. Parallel Offering Problem

The second setting we study in this paper is the Parallel Offering problem (par). Again, consider the case of a firm that has to hire candidates to fill k positions. Instead of conducting interviews, the firm has to send offers to candidates. The positions of the firm are now not identical, and for each candidate $i \in [n]$ and position $j \in [k]$ the firm knows v_{ij}: the reward collected by the firm if candidate i accepts an offer for position j, and p_{ij}: the probability that the candidate i accepts an offer for position j. The firm can carry out at most T offering rounds. At each round, the firm can send an offer for each unfilled position in parallel. Each candidate can receive at most one offer in total. The goal of the firm is to maximize the expectation of the sum of the values of the accepted offers. An instance for this problem is defined by a tuple $I = (k, T, n, p, v)$, where $1 \leq k \leq T \leq n$, $p \in [0,1]^{n\times k}$ and $v \in \mathbb{R}_+^{n\times k}$. For this problem, we define a policy as a function π that maps the remaining number of time steps, the remaining set of unfilled positions, and the set of candidates that have not yet received an offer, to an assignment of a subset of not-yet-offered candidates to unfilled positions. Let $\mathcal{I}_{\mathtt{par}}$ be the set of all instances and $\Pi^{\mathtt{par}}$ be the set of all policies for the Parallel Offering model. Again let $R_\pi(I)$ denote the expected reward of using policy $\pi \in \Pi^{\mathtt{par}}$ on instance $I \in \mathcal{I}_{\mathtt{par}}$. Let $\mathsf{OPT}_{\mathtt{par}}(I) := \sup_{\pi\in\Pi^{\mathtt{par}}} R_\pi(I)$ be the best possible expected reward that can be obtained from an instance $I \in \mathcal{I}_{\mathtt{par}}$. We say that a policy $\pi \in \Pi^{\mathtt{par}}$ is an α-approximation if

$$\inf_{I\in\mathcal{I}_{\mathtt{par}}} \frac{R_\pi(I)}{\mathsf{OPT}_{\mathtt{par}}(I)} \geq \alpha.$$

For the purpose of analysis, we can imagine that before the offering rounds start, each candidate has already decided which offers they would accept. We allow for these decisions to be correlated across different positions for a single candidate, but we assume independence across candidates.

In this context, we define a nonadaptive policy as follows. For each position, a list of candidates is constructed in a way such that a candidate cannot appear in more than one list and that each list contains no more than T candidates. Each list must also have a specified fixed ordering of its candidates. For each position, offers are sequentially sent to the candidates in their corresponding list, in its corresponding order, until either one of the candidates accepts or the list ends without any acceptance (independent of what happens with the other positions). The algorithm we develop for this problem falls under this definition, and thus we establish a lower bound on how well such nonadaptive policies can perform.

3.3. Simultaneous Offering Problem

The third and last setting we study is the Simultaneous Offering problem (sim). As in the previous models we study, a firm faces the challenge of filling k positions out of a pool of n candidates. There are two aspects that make this problem different from ptk and par. First, all offers must be made at the same time, and only once (hence the name Simultaneous Offering). The second aspect making this problem different is that the firm can end up hiring over capacity, but pays a linear cost for each candidate hired beyond k.[1]

Formally, we have the following problem. There are n candidates. For each candidate $i \in [n]$, we know p_i: the probability that candidate i accepts an offer if sent, and v_i: the value obtained if i accepts the offer. The firm must select a subset $S \subseteq [n]$ of candidates to send offers to, which has no cardinality constraint. When the subset S is decided, an offer is sent to each of the candidates in S. Each candidate accepts the offer independently with probability p_i, in which case they add value v_i to the firm. For each candidate that exceeds the capacity k, the firm incurs a linear cost of $c > 0$. Let $A \subseteq S$ be the (random) subset of candidates that accepted their offers. The reward obtained by the firm is $\sum_{i\in A} v_i - c \cdot [|A| - k]^+$. By rescaling values v_i by $1/c$, we can assume without loss of generality that $c=1$. The rescaled values v_i can be interpreted as the value that a candidate adds to the firm, relative to the cost of hiring a candidate over capacity.

With this notation, an instance of sim can be described by a tuple $I=(k,n,p,v)$, where $1 \le k \le n$, $p \in [0,1]^n$ and $v \in \mathbb{R}_+^n$. We define a policy as a function π that maps the problem instance I to a (possibly random) subset of candidates $S \subseteq [n]$. Let $\mathcal{I}_{\mathtt{sim}}$ denote the set of all instances and $\Pi^{\mathtt{sim}}$ denote the set of all policies for the Simultaneous Offering problem. Let $R_\pi(I)$ denote the expected reward of using policy $\pi \in \Pi^{\mathtt{sim}}$ on instance $I \in \mathcal{I}_{\mathtt{sim}}$. Let $\mathsf{OPT}_{\mathtt{sim}}(I)$ be the best possible expected reward that can be obtained from an instance $I \in \mathcal{I}_{\mathtt{sim}}$. We call a policy π an α-approximation if

$$\inf_{I \in \mathcal{I}_{\mathtt{sim}}} \frac{R_\pi(I)}{\mathsf{OPT}_{\mathtt{sim}}(I)} \ge \alpha.$$

One thing worth noting is that, although the objective function of the problem is submodular, it is not monotone and can take negative values. Thus, we cannot use general results for approximating maximization problems with submodular objective functions.

4. ProbeTop-k Problem

In this section, we provide our algorithm and analysis for the ProbeTop-k problem, where applicants are sequentially interviewed. We develop a nonadaptive, committed policy that achieves a $(1-e^{-k}k^k/k!)$ approximation of any (adaptive, noncommitted) policy. The algorithm first solves an LP relaxation that upper bounds the performance of any (adaptive, noncommitted) policy and uses then the LP solution as an input to decide the applicants that will be interviewed, in which order, and whether to hire an interviewed applicant, given their value.

In Section 4.1, we state the LP in question. In Section 4.2, we introduce a simple dependent rounding scheme that will be used in our approximation algorithm. In Sections 4.3 and 4.4, we introduce and analyze our approximation algorithm. In Section 4.5, we explain how to use our approximation algorithm as a subroutine for obtaining a PTAS. In Section 4.6, we treat the special case where values are distributed as weighted Bernoulli random variables and extend our results to the Sequential Offering problem. The analyses in this section, as well as those in Section 5, make use of correlation gap results by Yan (2011) and dependent rounding schemes by Gandhi et al. (2006). We recommend readers who are not familiar with these papers to visit Online Appendix A, which contains a summary of the relevant technical results contained in the previously mentioned papers, before reading the proofs. Throughout this section (except for Section 4.6), we use the word applicant indistinctly from candidate to avoid any confusion, although we had previously distinguished that an applicant becomes a candidate after being interviewed.

4.1. Linear Program

Consider the linear program $\mathrm{LP}_{\mathtt{ptk}}$, which for any instance of ProbeTop-k, upper bounds the performance of all possible policies. Linear programs in this spirit have been used exhaustively in the literature. Indeed, this relaxation is used to approach the same problem in Segev and Singla (2021), and it consists of a special case of the linear program used in Gupta and Nagarajan (2013). In $\mathrm{LP}_{\mathtt{ptk}}$, variable y_i is to be interpreted as the probability that applicant i is interviewed. The variable x_{ij} is to be interpreted as the probability that i is hired when $V_i = r_j$.

$$\begin{aligned} LP_{\mathtt{ptk}}(I) = \max \ & \sum_{i=1}^{n}\sum_{j=1}^{J} r_j x_{ij} \\ \text{s.t.} \ & x_{ij} \le y_i q_{ij} \qquad \forall i \in [n], j \in [J] \qquad (1) \\ & \sum_{i=1}^{n} y_i \le T \qquad (2) \\ & \sum_{i=1}^{n}\sum_{j=1}^{J} x_{ij} \le k \qquad (3) \\ & x_{ij} \ge 0 \qquad \forall i \in [n], j \in [J] \\ & 0 \le y_i \le 1 \qquad \forall i \in [n]. \end{aligned}$$

We first show that $\mathrm{LP}_{\mathtt{ptk}}$ upper bounds the optimal (adaptive, noncommitted) algorithm.

Lemma 1. *For any instance $I \in \mathcal{I}_{\mathtt{ptk}}$, we have $\mathrm{LP}_{\mathtt{ptk}}(I) \ge \mathsf{OPT}_{\mathtt{ptk}}(I)$.*

Although this result is not new, we provide a proof of the claim in Online Appendix B1 for completeness.

The following lemma establishes a convenient fact about the basic feasible solutions of the LP. This will let us develop a simple rounding scheme for selecting applicants to be interviewed.

Lemma 2. *Let $I \in \mathcal{I}_{\mathtt{ptk}}$ and let $y=(y_i)_{i\in[n]}, x=(x_{ij})_{i\in[n],j\in[J]}$ be a basic feasible solution of $\mathrm{LP}_{\mathtt{ptk}}(I)$. Then y has at most two noninteger components. If it has two noninteger components, they sum to one.*

The proof of this lemma is deferred to Online Appendix B2.

4.2. Dependent Rounding

We develop a simple dependent rounding scheme (which we call DR for short) that will be used as a subroutine of our approximation algorithm. DR receives an optimal solution $y=(y)_{i\in[n]}$ of $\mathrm{LP}_{\mathtt{ptk}}$ as an input and returns a (possibly random) vector $Y \in \{0,1\}^n$. DR works differently depending on the number of fractional components of the optimal solution y. In any case, for all i such that $y_i \in \{0,1\}$, it sets $Y_i = y_i$ with probability 1. If y

is integral, then Y is deterministic. If there is exactly one fractional component i', then it sets $Y_{i'} = 1$ with probability $y_{i'}$ (and it sets $Y_{i'} = 0$ otherwise). If there are exactly two fractional components i_1 and i_2, then it sets $Y_{i_1} = 1, Y_{i_2} = 0$ with probability y_{i_1} and sets $Y_{i_1} = 0, Y_{i_2} = 1$ with probability $1 - y_{i_1} = y_{i_2}$. It is easy to see that the output of DR satisfies the following two properties: (P1) $\mathbb{E}(Y_i) = y_i$ for every $i \in [n]$ and (P2) $\sum_{i\in[n]} Y_i \le T$ with probability 1.

4.3. Approximation Algorithm

We propose the following algorithm for the ProbeTop-k problem, which we call $\mathsf{ALG}_{\mathtt{ptk}}$. Given an instance, we first solve $\mathrm{LP}_{\mathtt{ptk}}(I)$. Let x, y be an optimal solution. Define

$$p_i = \frac{\sum_{j=1}^{J} x_{ij}}{y_i}, \quad v_i = \frac{\sum_{j=1}^{J} r_j x_{ij}}{\sum_{j=1}^{J} x_{ij}}. \tag{4}$$

If any of the denominators are zero, then define these values as zero.[2] For reasons that will soon become clear, p_i is to be interpreted as the probability that we accept applicant i given that they get interviewed, and v_i is to be interpreted as the expected value of i given that they get hired. Assume that after solving the LP, we relabel the applicants so that $v_1 \ge v_2 \ge \cdots \ge v_n$.

The algorithm first chooses which applicant to interview and the order in which the interviews are going to take place. For the first task, we use the dependent rounding DR just introduced. Specifically, we feed vector y as an input to DR and obtain binary random variables $\{Y_i\}_{i\in[n]}$. The algorithm will choose to interview applicants i with $Y_i = 1$. For the second task, the algorithm will always interview applicants in decreasing order of v_i. On interviewing applicant i and observing their value r_j, the algorithm hires i with probability $x_{ij}/(y_i q_{ij})$.

4.4. Analysis of $\mathsf{ALG}_{\mathtt{ptk}}$

To analyze the algorithm, let Y_i be the indicator that applicant i is included in the list of applicants to receive an interview. Let Q_{ij} be the indicator of $V_i = r_j$. Let P_i be the indicator that applicant i would be hired if they were interviewed while still having unfilled positions. We colloquially refer to this event as i "making the cut." We have

$$\mathbb{E}(P_i) = \sum_{j=1}^{J} \mathbb{E}(P_i | Q_{ij}) q_{ij} = \sum_{j=1}^{J} \frac{x_{ij}}{q_{ij} y_i} q_{ij} = p_i.$$

Define $Z_i = Y_i \cdot P_i$ as the indicator that i is included in the list of applicants to receive an interview and that i makes the cut. Note that Y_i is independent of P_i. Let $N_{i-1} = \sum_{\ell=1}^{i-1} Z_\ell$ be the number of applicants that are interviewed before applicant i that would be hired if there are still positions available. We can therefore write the (random) reward of our algorithm as $\sum_i V_i \mathbb{1}\{N_{i-1} < k\} Z_i$. The following lemma allows us to write the expected reward of $\mathsf{ALG}_{\mathtt{ptk}}$ in a way that will allow us to relate it to weighted rank functions of k-uniform matroids.

Lemma 3. *For any $I \in \mathcal{I}_{\mathtt{ptk}}$, we have $R_{\mathsf{ALG}_{\mathtt{ptk}}}(I) = \sum_{i=1}^n v_i \mathbb{E}(\mathbb{1}\{N_{i-1} < k\} Z_i)$.*

The proof of this lemma is deferred to Online Appendix B3. Because the applicants are labeled such that $v_1 \ge v_2 \ge \cdots \ge v_n$, Lemma 3 implies that our algorithm has the same expected reward as an algorithm that would first sample $Z_i = Y_i P_i$ for all applicants and then collect the k highest values v_i among those applicants with $Z_i = 1$. This interpretation is only possible because $\mathsf{ALG}_{\mathtt{ptk}}$ is committed. The same expressions can be derived for an algorithm that instead of sampling Y_i using DR, samples $\tilde{Y}_i$ with the same marginal probabilities as Y_i, but independently. These expressions are useful for showing that our algorithm outperforms a *hypothetical algorithm* that decides to interview each applicant i using the independent indicators $\tilde{Y}_i$ instead of the correlated indicators Y_i, potentially interviewing $T + 1$ applicants. The correlation induced by the constraint of T interviews on the actual algorithm only works in our favor.

Lemma 4. *Let $y = (y_i)_{i\in[n]}, x = (x_{ij})_{i\in[n], j\in[J]}$ be a basic feasible solution of $\mathrm{LP}_{\mathtt{ptk}}(I)$ with two fractional components i_1 and i_2. Let $\tilde{Y}_i$ be independent Bernoulli random variables with mean y_i. Define $\tilde{Z}_i = P_i \tilde{Y}_i$ and $\tilde{N}_i = \min\{k, \sum_{\ell=1}^{i} \tilde{Z}_\ell\}$. Then*

$$\sum_{i=1}^{n} v_i \mathbb{E}(Z_i \mathbb{1}\{N_{i-1} < k\}) \ge \sum_{i=1}^{n} v_i \mathbb{E}(\tilde{Z}_i \mathbb{1}\{\tilde{N}_{i-1} < k\}). \tag{5}$$

The proof of this lemma is deferred to Online Appendix B4.

To conclude our analysis, we relate the performance of the hypothetical algorithm (with independent indicators) and the objective function of the LP, through the weighted rank function of k-uniform matroids. For a set $S \subseteq [n]$ we define the weighted rank function for the k-uniform matroid with weights v as $v^*(S) = \max_{R\subseteq[S],\ |R|\le k} \sum_{i\in R} v_i$.

Both the reward of the hypothetical independent algorithm and the objective of $\mathrm{LP}_{\mathtt{ptk}}$ can be expressed as expectations of weighted rank functions of k-uniform matroids. We use this together with the correlation gap results by Yan (2011) to show the main result of this section.

Theorem 1. *For any instance $I \in \mathcal{I}^k_{\mathtt{ptk}}$, we have $R_{\mathsf{ALG}_{\mathtt{ptk}}}(I) \ge \left(1 - \frac{e^{-k}k^k}{k!}\right) \mathrm{LP}_{\mathtt{ptk}}(I)$.*

The proof of this theorem is deferred to Online Appendix B5. In Online Appendix B6, we show that the guarantee that our algorithm attains is tight.

It is worth noting that this algorithm can be de-randomized. Indeed, the algorithm will randomize between at most two fixed orders, so both of them can be evaluated and the best among them can be selected. Thus, we have a deterministic, nonadaptive, committed algorithm whose expected reward will be at least a $(1 - e^{-k}k^k/k!)$ factor of the expected reward of the optimal algorithm. Because our algorithm is nonadaptive and committed, it establishes a lower bound on how good an approximation factor can be achieved by restricting to these classes of policies.

4.5. PTAS for ProbeTop-*k*

We now explain how to combine $\mathsf{ALG}_{\texttt{ptk}}$ with the (adaptive, noncommitted) approximation algorithm proposed by Fu et al. (2018) to obtain a PTAS for ProbeTop-*k*. We know there is a $(1 - \varepsilon)$-approximation algorithm for ProbeTop-*k* whose runtime is exponential in *k* (Fu et al. 2018, theorem 4.2). The idea behind the combined PTAS is that for *k* large enough, $\mathsf{ALG}_{\texttt{ptk}}$ obtains an approximation factor better than $1 - \varepsilon$, so for small values of *k* we would run the algorithm by Fu et al. (2018), and for large values of *k* we would run our $\mathsf{ALG}_{\texttt{ptk}}$. This essentially allows us to treat *k* as a constant in the algorithm of Fu et al. (2018). Our algorithm has runtime polynomial in *k*, but it is without loss to assume that *k* is input in unary because if *k* (capacity) exceeds the number of given distributions (applicants/candidates), then the problem is trivial.

Specifically, for $\varepsilon \in (0, 1/e)$, let k^* be the smallest $k \geq 1$ such that $\varepsilon > e^{-k}k^k/(k!)$.[3] For $k < k^*$, the PTAS will run the algorithm by Fu et al. (2018) to obtain a $1 - \varepsilon$ approximation in polynomial time because k^* is a constant. For $k \geq k^*$, the PTAS will run $\mathsf{ALG}_{\texttt{ptk}}$ and obtain, in polynomial time, an approximation factor of $1 - e^{-k}k^k/(k!) > 1 - \varepsilon$.

Corollary 1. *There is a PTAS for ProbeTop-k.*

4.6. Weighted Bernoulli Values and the Sequential Offering Problem

ProbeTop-*k* is closely related to the Sequential Offering problem (`seq`) studied by Purohit et al. (2019). In this problem, the hiring firm has already interviewed *n* candidates and must decide the order in which to send offers to them. Each candidate *i* has a probability q_i of accepting an offer and adds a value r_i to the firm if they accept an offer. The firm can hire at most *k* candidates and send at most *T* offers in total. This model is closely related to ProbeTop-*k* when considering the special case where the value of applicant *i* takes value r_i with probability q_i (representing acceptance of an offer), and zero with probability $1 - q_i$ (representing rejection of an offer). The subtlety making `ptk` different from `seq` is that policies for `ptk` can reject an applicant *i* even if they had a realization to value r_i, which is not allowed in `seq`. We show, however, that $\mathsf{ALG}_{\texttt{ptk}}$ can be forced to hire any applicant when their realized value is r_i without loss, therefore making it an admissible algorithm for the Sequential Offering model with the same guarantee.

Indeed, we can rewrite $\mathrm{LP}_{\texttt{ptk}}$ for this special case as

$$\begin{aligned} \max \quad & \sum_{i=1}^{n} r_i x_i \\ \text{s.t.} \quad & x_i \leq y_i q_i \quad \forall i \in [n], \quad \sum_{i=1}^{n} y_i \leq T, \quad \sum_{i=1}^{n} x_i \leq k, \\ & x_i \geq 0 \qquad \forall i \in [n], \quad 0 \leq y_i \leq 1 \quad \forall i \in [n], \end{aligned}$$

where x_i is to be interpreted as the probability of hiring applicant *i* and $V_i = r_i$ (we can omit the variable corresponding to $V_i = 0$ because it has a zero coefficient in the objective). The following lemma will let us restrict $\mathsf{ALG}_{\texttt{ptk}}$ to be admissible for the Sequential Offering problem.

Lemma 5. *For weighted Bernoulli instances,* $\mathrm{LP}_{\texttt{ptk}}$ *has an optimal solution with* $x_i = y_i q_i \ \forall i \in [n]$.

The proof of this lemma is deferred to Online Appendix B7.

Recall that after interviewing *i*, $\mathsf{ALG}_{\texttt{ptk}}$ will hire them with probability $x_i/(y_i q_i)$. Therefore, if we restrict to solutions with $x_i = y_i q_i$, then $\mathsf{ALG}_{\texttt{ptk}}$ will hire applicant *i* with probability 1 if $V_i = r_i$, making it admissible for the Sequential Offering problem.

For this specific family of instances, Equations (4) yield

$$p_i = \frac{x_i}{y_i} = x_i \frac{q_i}{x_i} = q_i, \quad v_i = \frac{r_i x_i}{x_i} = r_i.$$

We can use these expressions to further simplify the LP by removing variables x_i and express it in terms of p_i and v_i, leading to $\mathrm{LP}_{\texttt{seq}}$.

$$\begin{aligned} \mathrm{LP}_{\texttt{seq}}(I) = \max \quad & \sum_{i=1}^{n} v_i y_i p_i \\ \text{s.t.} \quad & \sum_{i=1}^{n} y_i \leq T, \quad \sum_{i=1}^{n} y_i p_i \leq k, \quad 0 \leq y_i \leq 1 \\ & \forall i \in [n]. \end{aligned}$$

Being consistent with the previously introduced notation, let $\mathcal{I}_{\texttt{seq}}$ be the set of all instances for the Sequential Offering problem. Let $\mathcal{I}^k_{\texttt{seq}}$ be the set of all instances of `seq` that have exactly *k* positions to fill. Let $\mathsf{ALG}_{\texttt{seq}}$ be the modified version of $\mathsf{ALG}_{\texttt{ptk}}$ that, when faced with instances of weighted Bernoulli random variables, modifies the solution of the LP as in Lemma 5 such that the algorithm is admissible for `seq`. We then have the following corollary of Theorem 1.

Corollary 2. *For any instance* $I \in \mathcal{I}^k_{\texttt{seq}}$, *we have* $R_{\mathsf{ALG}_{\texttt{seq}}}(I) \geq \left(1 - \frac{e^{-k}k^k}{k!}\right) \mathrm{LP}_{\texttt{seq}}(I)$.

5. Parallel Offering

We turn to the Parallel Offering model defined in Section 3.2. We provide an algorithm that achieves a $(1 - 1/e)$ approximation of the optimal policy. The algorithm works by solving a LP relaxation and rounding its solution to decide who to offer which position, and in which order.

In Section 5.1, we introduce the LP in question. In Section 5.2, we present our algorithm, and in Section 5.3, we provide its analysis. In Section 5.4, we study the special case with identical positions introduced by Purohit et al. (2019) and establish a connection between the Parallel and Sequential Offering problems.

5.1. Linear Program

For an instance $I \in \mathcal{I}_{\texttt{par}}$, we introduce $\text{LP}_{\texttt{par}}(I)$, with LP variables y_{ij} for $i \in [n]$ and $j \in [k]$. Variable y_{ij} is to be interpreted as the probability that candidate i receives an offer for position j. As with $\text{LP}_{\texttt{ptk}}$, this LP only enforces that the problem's constraints are satisfied in expectation.

$$\text{LP}_{\texttt{par}}(I) = \max \quad \sum_{j=1}^{k}\sum_{i=1}^{n} v_{ij} y_{ij} p_{ij}$$

$$\text{s.t.} \quad \sum_{i=1}^{n} y_{ij} \le T \qquad \forall j \in [k] \qquad (6)$$

$$\sum_{i=1}^{n} y_{ij} p_{ij} \le 1 \qquad \forall j \in [k] \qquad (7)$$

$$\sum_{j=1}^{k} y_{ij} \le 1 \qquad \forall i \in [n] \qquad (8)$$

$$0 \le y_{ij} \le 1 \qquad \forall (i,j) \in [n] \times [k].$$

We start by formally proving that $\text{LP}_{\texttt{par}}$ upper-bounds the expected reward of any algorithm.

Lemma 6. *For any instance $I \in \mathcal{I}_{\texttt{par}}$, we have $\text{LP}_{\texttt{par}}(I) \ge \mathsf{OPT}_{\texttt{par}}(I)$.*

The proof of this lemma is deferred to Online Appendix C1.

5.2. Approximation Algorithm

We now present our algorithm for the Parallel Offering model, which we refer to as $\mathsf{ALG}_{\texttt{par}}$. The algorithm first solves $\text{LP}_{\texttt{par}}$ to produce an optimal solution y. With the solution at hand, the algorithm will round it to obtain a random binary matrix $Y = (Y_{ij})_{(i,j) \in [n] \times [k]}$. To round our solution here, we use the dependent rounding scheme developed by Gandhi et al. (2006) that is described in Online Appendix A2. Specifically, the nodes of one side of the bipartite graph are the n applicants, and the nodes of the other side are the k positions. The weight of each edge $(i,j) \in [n] \times [k]$ is the fractional value of y_{ij} from the optimal solution of $\text{LP}_{\texttt{par}}$. After the solution is rounded, the algorithm uses the rounded solution to make a sequential offering list for each position so that candidate i is included in the list for position j if $Y_{ij} = 1$. The properties of the dependent rounding scheme by Gandhi et al. (2006) combined with the constraints of $\text{LP}_{\texttt{par}}$ will ensure that (a) each list has at most T candidates, and (b) each candidate is included in at most one list. After the lists are formed, all lists are run in parallel, and the order in which candidates i for each position j are sent offers is in decreasing order of v_{ij}.

5.3. Analysis of $\mathsf{ALG}_{\texttt{par}}$

For the analysis, let L_j denote the expected value of the candidate that ends up being hired for position j. Define $Z_{ij} = P_{ij} Y_{ij}$ and $\tilde{Z}_{ij} = P_{ij} \tilde{Y}_{ij}$, where $(\tilde{Y}_{ij})_{i \in [n], j \in [k]}$ are independent Bernoulli random variables with $\mathbb{E}(\tilde{Y}_{ij}) = y_{ij}$. Let $D_j = \{i : Z_{ij} = 1\}$ and $\tilde{D}_j = \{i : \tilde{Z}_{ij} = 1\}$. Let $v_j^*(S) = \max_{i \in S} v_{ij}$. It is clear to see that $L_j = \mathbb{E}(v_j^*(D))$.

The first step for showing the guarantee of $\mathsf{ALG}_{\texttt{par}}$ is to show that the reward collected by a list is not lower than what we would collect if we rounded each component of y independently. This is a consequence of the negative correlation property (P3) of the dependent rounding scheme by Gandhi et al. (2006). This result is formalized in the following lemma.

Lemma 7. *For all $j \in [k]$, we have $\mathbb{E}(v_j^*(D)) \ge \mathbb{E}(v_j^*(\tilde{D}))$.*

The proof of this lemma is deferred to Online Appendix C2. Lemma 7 analyzes each position in isolation, even if two or more positions are identical. In fact, our proof does not appear to extend to multiple identical positions. Indeed, our proof only makes use of properties (P1) and (P3) from the dependent rounding scheme of Gandhi et al. (2006). We prove in Online Appendix C3 that only using these properties is insufficient for proving an analogue of Lemma 7 for multiple positions.

Continuing with the main result, let L_j^* denote $\sum_{i=1}^{n} v_{ij} y_{ij} p_{ij}$, so that the objective function of $\text{LP}_{\texttt{par}}$ can be expressed as $\sum_{j=1}^{k} L_j^*$. The following lemma helps establish the desired bound for each separate list; at most, one candidate is hired from each list. Both the correlation gap results and the rounding scheme mentioned in Online Appendix A are used in our analysis.

Lemma 8. *For all $j \in [k]$, we have $\mathbb{E}(v_j^*(\tilde{D})) \ge (1 - 1/e) L_j^*$.*

The proof of this lemma is deferred to Online Appendix C4.

By combining Lemmas 7 and 8 we obtain the main result of the section.

Theorem 2. *For any instance $I \in \mathcal{I}_{\texttt{par}}$, we have $R_{\mathsf{ALG}_{\texttt{par}}}(I) \ge (1 - 1/e)\text{LP}_{\texttt{par}}(I)$.*

In Online Appendix C5, we establish that the bound achieved by our algorithm is tight.

5.4. Identical Positions and the Cost of Batching

We turn to the special case where all positions are identical (i.e., $v_{ij} = v_i$ and $p_{ij} = p_i$ for all $i \in [n]$ and $j \in [k]$). For this case, we can obtain a connection between the Sequential Offering model and the Parallel Offering model through their respective LPs.

Let $\mathcal{I}^{=}_{\texttt{par}} \subseteq \mathcal{I}_{\texttt{par}}$ be the set of instances of par in which all positions are identical. For a given instance $I \in \mathcal{I}^{=}_{\texttt{par}}$, construct $I' \in \mathcal{I}_{\texttt{seq}}$ with the same candidates as in I, but with a budget of kT sequential offers (instead of T parallel offering rounds). We obtain the following lemma.

Lemma 9. *For any* $I \in \mathcal{I}^{=}_{\texttt{par}}$, *we have* $\mathrm{LP}_{\texttt{par}}(I) = \mathrm{LP}_{\texttt{seq}}(I')$.

The proof of this lemma is deferred to Online Appendix C6.

Lemma 9 implies the following corollary, which we refer to as the cost of batching.

Corollary 3. *For any* $I \in \mathcal{I}^{=}_{\texttt{par}}$, *we have* $R_{\mathsf{ALG}_{\texttt{par}}}(I) \geq (1 - 1/e)\mathsf{OPT}_{\texttt{seq}}(I')$.

This corollary helps us understand how costly it can be to send offers in batches instead of one by one like in the Sequential Offering problem. By reducing to T parallel offering rounds instead of kT sequential offering rounds, we know that we cannot be worse by more than a factor of $(1 - 1/e)$.

6. Simultaneous Offering

In this section, we study the Simultaneous Offering problem described in Section 3.3. We are interested in a class of "value-ordered" policies. We develop such a policy that, when faced with instances where all values of candidates are lower-bounded by some $\tau \in (0, 1)$, achieves an α_k^τ-approximation, where α_k^τ is an increasing function that maps the number of positions $k \in \mathbb{N}$ and the lower bound τ to a real number between zero and one. Our algorithm first solves an LP relaxation. It then uses a modification of the optimal solution to decide which candidates will receive offers.

In Section 6.1, we define the class of value-ordered policies, which we show can perform arbitrarily poorly without the assumption of the lower bound τ. In Section 6.2, we introduce an LP relaxation that is used in our algorithm. In Sections 6.3 and 6.4, we introduce and analyze our algorithm, providing lower bounds on how well value-ordered policies can perform.

6.1. Value-Ordered Policies

For this problem, we focus on a natural class of policies, which we call *value-ordered policies*. Assume that $v_1 \geq v_2 \geq \cdots \geq v_n$. A value-ordered policy is defined by an integer $m \in [n]$ and sends offers to the m candidates with the highest values (i.e., $S = \{1, \ldots, m\}$). This family of policies is practically well motivated because a firm generally does not want to withhold sending offers to high-value candidates whom it deems "too good" for itself. The optimal value-ordered policy also can be obtained efficiently by solving a dynamic program (see Online Appendix D1 for details).

As natural as they seem, value-ordered policies can achieve an arbitrarily poor approximation factor. Consider the following simple example.

Example 1. Consider an instance with $k = 1$ and $n = 2$. For small ε, candidate 1 has $v_1 = p_1 = \varepsilon$, while candidate 2 has $v_2 = \varepsilon(1 - \varepsilon)$ and $p_2 = 1$. There are three possible policies for this instance: $\{1, 2\}$, $\{1\}$, and $\{2\}$. The two first are value-ordered policies. The reward for using $\{1, 2\}$ is zero, for using $\{1\}$ is ε^2, and for using $\{2\}$ is $\varepsilon(1 - \varepsilon)$. For $\varepsilon < 1/2$, this means that the optimal policy is $\{2\}$, and the optimal value-ordered policy is $\{1\}$. The approximation factor achieved by value-ordered policies in this instance is equal to $\varepsilon/(1 - \varepsilon)$, which can be made arbitrarily small by taking $\varepsilon \to 0$.

A similar example can be constructed given any amount of positions k, as we show later in Theorem 4. The intuition behind the previous example is that when ε is small, adding a candidate over capacity provides negligible benefit compared to cost, making the capacity k almost a hard constraint. Although the difficulties of this specific example can be avoided by ordering the candidates in descending order of $v_i p_i$ instead of v_i, or using the greedy policy which starts with an empty set of candidates and iteratively adds the candidate with the highest marginal benefit to the set, we show that these alternate algorithmic ideas can also achieve an arbitrarily poor approximation in Online Appendix D2.

We hereafter aim for parametric guarantees for value-ordered policies under the assumption that the values of the candidates are lower-bounded by a parameter $\tau > 0$. Let $\mathcal{I}^{\tau}_{\texttt{sim}} = \{I \in \mathcal{I}_{\texttt{sim}} : v_i \geq \tau \ \ \forall i \in [n]\}$ be the set of τ-bounded instances. Because we normalized c (the cost of hiring each candidate over capacity) to be one, the value of τ can be interpreted as how hard or soft the capacity constraint of k is in practice. For example, if τ is close to one, then the capacity constraint can be viewed as soft because the cost of hiring over capacity would be almost fully compensated by the value of any candidate. If the capacity constraint is hard in practice, then τ could be close to zero.

We remark that if τ is only small because of "irrelevant" low-value candidates who would never receive an offer, then it does not negatively affect our results. More precisely, our guarantee depends on the lowest value of a candidate with positive mass in the solution of the LP introduced in Section 6.2. We ignore this distinction in our definition of τ-bounded instances for simplicity.

We say that a policy π is an α-approximation for τ-bounded instances if

$$\inf_{I\in\mathcal{I}^{\tau}_{\mathtt{sim}}} \frac{R_\pi(I)}{\mathsf{OPT}_{\mathtt{sim}}(I)} \geq \alpha.$$

If $\tau \geq 1$, then the problem is trivial: It is optimal to send an offer to every candidate. We will therefore restrict our attention to $\tau \in (0,1)$.

Before we carry on, we define $V_{\mathtt{high}} = \{i \in [n] : v_i > 1\}$ to be the set of candidates whose value is greater than one. These are candidates that we would like to hire even if we know they would violate capacity, essentially making the number of positions random, as they will all receive offers but it is uncertain how many would accept. To our understanding, there is no easy way to reduce the problem to one where they do not exist, when both $V_{\mathtt{high}}$ and $[n]\setminus V_{\mathtt{high}}$ are nonempty.

6.2. LP Relaxation

To obtain approximation factors for value-ordered policies, we introduce a linear programming relaxation of $\mathsf{OPT}_{\mathtt{sim}}$, which we call $\mathrm{LP}_{\mathtt{sim}}$:

$$\begin{aligned}
\mathrm{LP}_{\mathtt{sim}}(I) = \max_{y,z} & \sum_{i\in[n]} v_i p_i y_i + z\\
\text{s.t.} \quad & z \leq k - \sum_{i\in[n]} p_i y_i\\
& z \leq 0\\
& 0 \leq y_i \leq 1 \quad \forall i \in [n].
\end{aligned}$$

As with the previous linear programs, y_i is to be interpreted as the probability that candidate i receives an offer. This linear program optimizes over randomized policies that pay a penalty for the difference between the expected number of candidates hired and k rather than the realized number of candidates hired in excess of k. Thus, by applying Jensen's inequality we can show that $\mathrm{LP}_{\mathtt{sim}}$ is an upper bound of $\mathsf{OPT}_{\mathtt{sim}}$.

Lemma 10. *For any $I \in \mathcal{I}_{\mathtt{sim}}$, we have $\mathrm{LP}_{\mathtt{sim}}(I) \geq \mathsf{OPT}_{\mathtt{sim}}(I)$.*

The proof of this lemma is deferred to Online Appendix D3.

We proceed to show a property about optimal solutions of $\mathrm{LP}_{\mathtt{sim}}$ that closely relates it to a value-ordered policy.

Lemma 11. *Let $I \in \mathcal{I}_{\mathtt{sim}}$. There exists an optimal solution (y, z) of $\mathrm{LP}_{\mathtt{sim}}(I)$ with an index j such that $y_i = 1$ for $1 \leq i \leq j-1$, $y_j > 0$, and $y_i = 0$ for $j+1 \leq i \leq n$.*

The proof of this lemma follows from the fact that, for any fixed z, $\mathrm{LP}_{\mathtt{sim}}$ corresponds to an instance of the fractional knapsack problem. It is well known that optimal solutions to this problem satisfy the structure described in Lemma 11 (Goodrich and Tamassia 2001, chapter 5), so it is true for any optimal choice of z. Given Lemma 11, we can easily construct a randomized value-ordered policy from an optimal solution of $\mathrm{LP}_{\mathtt{sim}}$. Indeed, we could simply send an offer to each candidate i independently with probability y_i. This policy randomizes between two value-ordered policies: sending offers to candidates $\{1,\ldots,j\}$ or to candidates $\{1,\ldots,j-1\}$. We will make use of this idea when constructing our actual approximation algorithm.

An important object in the definition and analyses of our policies is the "total mass" of a solution of $\mathrm{LP}_{\mathtt{sim}}$. For a feasible solution y, its total mass is given by $\sum_{i\in[n]} y_i p_i$. This can be interpreted as the expected number of candidates that would be hired if we were to send an offer to each candidate i with probability y_i. Lemma 11 also implies that an optimal solution is completely determined by its total mass. This is because the solution can be constructed by "filling" the components from smallest to largest index until we obtain the desired total mass.

The following lemma allows us to determine the total mass of optimal solutions of $\mathrm{LP}_{\mathtt{sim}}$ based on aggregate parameters of instances and will be useful later in the analysis.

Lemma 12. *Let $I \in \mathcal{I}_{\mathtt{sim}}$. Let (y, z) be an optimal solution of $\mathrm{LP}_{\mathtt{sim}}(I)$ that satisfies the structure given in Lemma 11. Let $P_{\mathtt{high}} = \sum_{i\in V_{\mathtt{high}}} p_i$ and $P_{\mathtt{total}} = \sum_{i\in[n]} p_i$. The following holds:*

1. *If $P_{\mathtt{high}} > k$, then $y_i = 1$ for all $i \in V_{\mathtt{high}}$ and $y_i = 0$ otherwise. Therefore, $\sum_{i\in[n]} y_i p_i = P_{\mathtt{high}}$;*
2. *If $P_{\mathtt{high}} \leq k$, then $\sum_{i\in[n]} y_i p_i = \min\{k, P_{\mathtt{total}}\}$.*

The proof of this lemma is deferred to Online Appendix D4.

6.3. Approximation Algorithm

Let us introduce our approximation algorithm for $\mathtt{sim}$, which we call $\mathsf{ALG}^s_{\mathtt{sim}}$. The policy takes as input an optimal solution y of $\mathrm{LP}_{\mathtt{sim}}$ and a parameter $s \in [0,1]$. The idea of the policy is to truncate the optimal solution of the LP. This truncation is done by scaling down the total mass of the optimal LP solution by a factor s and then using this mass to "fill" the new variables from smallest to largest index. We then use this truncated solution to decide which candidates will receive offers.

Formally, the first step is to construct an alternative solution y' by truncating y the following way. Let $\kappa = s\sum_{i\in[n]} y_i p_i$ be the total mass of the optimal LP solution, scaled down by a factor s. Let j be the first index such that $\sum_{i=1}^{j+1} p_i > \kappa$. We let $y'_i = 1$ for $1 \leq i \leq j$, $y'_i = 0$ for $j+2 \leq i \leq n$, and set y'_{j+1} such that $\sum_{i=1}^{j+1} y'_i p_i = \kappa$. After solution y' is constructed, the offers are sent independently to each candidate i with probability y'_i. Given the structure of y', $\mathsf{ALG}^s_{\mathtt{sim}}$ clearly randomizes between two value-ordered policies: sending offers to candidates 1 through j, and sending offers to candidates 1 through $j+1$.

For any k and τ, parameter s can be optimized to maximize the algorithm's performance, and we let s^* denote the optimal value. Specifically, s^* is defined as an optimal solution to the problem:

$$\sup_{0 \le s \le 1} \left(s - \frac{s}{\tau} + \frac{\mathbb{E}[\min\{\text{Pois}(sk), k\}]}{\tau k} \right).$$

We will use s^* only in our analysis to lower-bound the performance of the best value-ordered policy.

6.4. Analysis of $\mathsf{ALG}_{\texttt{sim}}$

The analysis of $\mathsf{ALG}^s_{\texttt{sim}}$ and $\mathsf{ALG}^{s^*}_{\texttt{sim}}$ is done in two cases. One case is $\sum_{i \in V_{\texttt{high}}} p_i \le k$, in which sending an offer to all candidates in $V_{\texttt{high}}$ hires an expected number of candidates less than or equal to k. The other case is $\sum_{i \in V_{\texttt{high}}} p_i > k$.

Case 1: $\sum_{i \in V_{\texttt{high}}} p_i \le k$. In this case, we can show a lower bound for the performance of $\mathsf{ALG}^s_{\texttt{sim}}$ that depends on the choice of s.

Lemma 13. *For any $s \in [0,1]$, $\tau \in (0,1)$, and $I \in \mathcal{I}^\tau_{\texttt{sim}}$ with $\sum_{i \in V_{\texttt{high}}} p_i \le k$, we have*

$$R_{\mathsf{ALG}^s_{\texttt{sim}}}(I) \ge \left(s - \frac{s}{\tau} + \frac{\mathbb{E}[\min\{\text{Pois}(sk), k\}]}{\tau k} \right) \text{LP}_{\texttt{sim}}(I).$$

The proof of this lemma is deferred to Online Appendix D5.

Case 2: $\sum_{i \in V_{\texttt{high}}} p_i > k$. In this case, we can show that for an optimally chosen parameter s^*, the guarantee obtained by $\mathsf{ALG}^{s^*}_{\texttt{sim}}$ cannot be worse than the previous case where $\sum_{i \in V_{\texttt{high}}} p_i \le k$. A key observation for analyzing this new case is that as Lemma 12 states, all candidates i whose value y_i in the optimal LP solution will have $v_i \ge 1$, so $s^* = 1$. Indeed, setting $s < 1$ can only decrease the chances of sending offers to candidates in $V_{\texttt{high}}$, and these are candidates that the firm would always want to hire (even over capacity). With this observation in hand, we prove the following, via the construction of an auxiliary instance in which the probabilities of acceptance are deflated.

Lemma 14. *Let $I = (k, n, p, v) \in \mathcal{I}^\tau_{\texttt{sim}}$ be an instance with $\sum_{i \in V_{\texttt{high}}} p_i > k$. Then there exists an instance $I' = (k, n, p', v) \in \mathcal{I}^\tau_{\texttt{sim}}$ such that $\sum_{i \in V_{\texttt{high}}} p'_i = k$ and*

$$\frac{R_{\mathsf{ALG}^{s^*}_{\texttt{sim}}}(I)}{\text{LP}_{\texttt{sim}}(I)} \ge \frac{R_{\mathsf{ALG}^{s^*}_{\texttt{sim}}}(I')}{\text{LP}_{\texttt{sim}}(I')}.$$

The proof of this lemma is deferred to Online Appendix D6.

By combining Lemmas 13 and 14 for an optimally chosen parameter s^*, we obtain the main result of this section.

Theorem 3. *For any $\tau \in (0,1)$ and $I \in \mathcal{I}^\tau_{\texttt{sim}}$, we have $R_{\mathsf{ALG}^{s^*}_{\texttt{sim}}}(I) \ge \alpha^\tau_k \text{LP}_{\texttt{sim}}(I)$, where*

$$\alpha^\tau_k = \sup_{0 \le s \le 1} \left(s - \frac{s}{\tau} + \frac{\mathbb{E}[\min\{\text{Pois}(sk), k\}]}{\tau k} \right).$$

The performance of $\mathsf{ALG}^{s^*}_{\texttt{sim}}$ clearly dominates that of $\mathsf{ALG}^1_{\texttt{sim}}$, where $\mathsf{ALG}^1_{\texttt{sim}}$ directly follows the LP solution. By Lemma 13, $\mathsf{ALG}^1_{\texttt{sim}}$ has a guarantee of at least $1 - \frac{1}{\tau} + \frac{\mathbb{E}[\min\{\text{Pois}(k), k\}]}{\tau k}$, which equals $1 - e^{-k}k^k/(k!\tau)$, as we derive in Online Appendix A3. By combining this last expression with Stirling's approximation, we obtain the following result about the asymptotic optimality of $\mathsf{ALG}^{s^*}_{\texttt{sim}}$ when the number of positions grows large and the values of candidates are bounded away from zero.

Corollary 4. *For any $\tau \in (0,1)$ and any instance $I \in \mathcal{I}^\tau_{\texttt{sim}}$, we have $\mathsf{ALG}^{s^*}_{\texttt{sim}} \ge (1 - O(1/\sqrt{k}))\text{LP}_{\texttt{sim}}(I)$, where k is the number of positions in the instance.*

We now provide an upper bound for the guarantee that can be achieved using value-ordered policies. We construct an instance consisting of $n = j^2 + k$ candidates[4], with j being a (large) integer. The first j^2 candidates are of type 1, who have $p_i = k/j$ and $v_i = \tau + \delta$, where $\delta > 0$ is small. The remaining k candidates are of type 2, who have $p_i = 1$ and $v_i = \tau$. The idea behind this result is that any optimal value-ordered policy will only send offers to type 1 candidates. Conversely, the optimal policy does not perform worse than a policy that sends offers to k type 2 candidates and zero type 1 candidates.

Theorem 4. *For any $\varepsilon > 0$ and $\tau \in (0,1)$, there exists an instance $I \in \mathcal{I}^\tau_{\texttt{sim}}$ such that no value-ordered policy can have an expected reward greater than $(\beta^\tau_k + \varepsilon)\mathsf{OPT}_{\texttt{sim}}(I)$, where*

$$\beta^\tau_k = \sup_{s \ge 0} \left(s - \frac{s}{\tau} + \frac{\mathbb{E}[\min\{\text{Pois}(sk), k\}]}{\tau k} \right).$$

The proof of this theorem is deferred to Online Appendix D7. In Online Appendix D8, we analyze the region in which our bounds coincide, together with plots showing α^τ_k and β^τ_k for several values of k and τ.

7. Future Directions

We close the paper by pointing out natural follow-up research questions that arise from our work. An interesting open direction is to combine the sequential interviewing problem with the offering problem. This can be done by generalizing the sequential interviewing problem to a setting where candidates are not assured to accept offers and instead have a probability of accepting which might depend on factors such as their realized value. The firm can, at the end of each period, decide to send offers to any candidate who has already been interviewed. Another open direction is to combine the

Simultaneous Offering problem with the Parallel Offering problem, as implicitly suggested in Purohit et al. (2019). In this setting, the firm could, at each round, send more offers than positions remaining and face the risk of hiring over capacity (at a cost). One last future direction on the modeling side is how the firm should behave when the problem parameters are inaccurate. In particular, acceptance probabilities can be very difficult to estimate, so developing algorithms that are robust to these parameters' misspecification could be of great interest to firms.

On the technical side, we believe the Simultaneous Offering problem introduced is a parsimonious new variant of overbooking. Although our focus was to analyze the performance of value-ordered policies, we do not know of any hardness or algorithmic results for finding the best offer set. It would be interesting if an optimal or near-optimal (i.e., PTAS) algorithm could be found. The question of whether we can improve on $1 - 1/e$ for the sequential offering problem (or maybe even obtain a PTAS) is also open.

Acknowledgments

The authors thank José Correa for insightful discussions about Sequential Offering, Rad Niazadeh for pointing us to the highly relevant reference (Bradac et al. 2019), and Aravind Srinivasan for insightful discussions about negative association. The authors further thank anonymous reviewers for *Operations Research* who gave exceptionally detailed comments and identified the corollaries in Sections 4.5 and 6.4. A one-page abstract of this work appeared in the 18th conference on *Web and Internet Economics* 2022.

Endnotes

[1] Requiring the firm to hire at most k candidates with probability 1 would make this problem trivial: the firm would send an offer to the k candidates with the highest expected value $v_i p_i$.

[2] If $y_i = 0$, then $\sum_{j=1}^{J} x_{ij} = 0$ too. If $y_i = 0$, then the algorithm will never interview applicant i, so the definition of these values is not relevant for the analysis.

[3] We know that k^* always exists because $e^{-k}k^k/(k!)$ is a decreasing function that equals $1/e$ when $k = 1$ and converges to zero as $k \to \infty$.

[4] We do not need exactly j^2 candidates of type 1 for showing the result. Any amount of candidates ℓ_j with $\ell_j/j \to \infty$ as $j \to \infty$ will suffice.

References

Agrawal S, Sethuraman J, Zhang X (2020) On optimal ordering in the optimal stopping problem. Biró P, Hartline J, eds. *Proc. 21st ACM Conf. on Econom. and Comput.* (Association for Computing Machinery, New York), 187–188.

Arnosti N, Ma W (2023) Tight guarantees for static threshold policies in the prophet secretary problem. *Oper. Res.* 71(5):1777–1788.

Beyhaghi H, Golrezaei N, Leme RP, Pál M, Sivan B (2021) Improved revenue bounds for posted-price and second-price mechanisms. *Oper. Res.* 69(6):1805–1822.

Bradac D, Singla S, Zuzic G (2019) (Near) optimal adaptivity gaps for stochastic multi-value probing. Achlioptas D, Végh LA, eds. *Approximation, Randomization, Combin. Optim. Algorithms Techniques*, Leibniz International Proceedings in Informatics (LIPIcs), vol. 145 (Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik, Wadern), 1–49.

Cominetti R, Correa JR, Rothvoß T, Martín JS (2010) Optimal selection of customers for a last-minute offer. *Oper. Res.* 58(4-part-1):878–888.

Esfandiari H, Hajiaghayi M, Liaghat V, Monemizadeh M (2017) Prophet secretary. *SIAM J. Discrete Math.* 31(3):1685–1701.

Fu H, Li J, Xu P (2018) A PTAS for a class of stochastic dynamic programs. Chatzigiannakis I, Kaklamanis C, Marx D, Sannella D, eds. *45th Internat. Colloquium Automation, Languages, Programming (ICALP 2018)*, Leibniz International Proceedings in Informatics (LIPIcs), vol. 107 (Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Wadern, Germany), 1–56.

Gallego G, Segev D (2022) A constructive prophet inequality approach to the adaptive probemax problem. Preprint, submitted October 14, https://arxiv.org/abs/2210.07556.

Gallego G, Topaloglu H (2019) *Revenue Management and Pricing Analytics*, vol. 209 (Springer, Berlin).

Gandhi R, Khuller S, Parthasarathy S, Srinivasan A (2006) Dependent rounding and its applications to approximation algorithms. *J. ACM* 53(3):324–360.

Goodrich MT, Tamassia R (2001) *Algorithm Design: Foundations, Analysis, and Internet Examples* (John Wiley & Sons, Hoboken, NJ).

Gupta A, Nagarajan V (2013) A Stochastic probing problem with applications. Goemans M, Correa J, eds. *Integer Programming and Combinatorial Optimization. IPCO 2013*, Lecture Notes in Computer Science, vol. 7801 (Springer, Berlin, Heidelberg).

Gupta A, Nagarajan V, Singla S (2016) Algorithms and adaptivity gaps for stochastic probing. Krauthgamer R, ed. *Proc. 27th Annual ACM-SIAM Sympos. on Discrete Algorithms* (Association for Computing Machinery, New York), 1731–1747.

Gupta A, Nagarajan V, Singla S (2017) Adaptivity gaps for stochastic probing: Submodular and xos functions. Krauthgamer R, ed. *Proc. 28th Annual ACM-SIAM Sympos. Discrete Algorithms* (SIAM, Philadelphia), 1688–1702.

Hill T (1983) Prophet inequalities and order selection in optimal stopping problems. *Proc. Amer. Math. Soc.* 88(1):131–137.

Kleinberg R, Weinberg SM (2012) Matroid prophet inequalities. Karloff H, ed. *Proc. 44th Annual ACM Sympos. Theory Comput.* (Association for Computing Machinery, New York), 123–136.

Krengel U, Sucheston L (1977) Semiamarts and finite values. *Bull. Amer. Math. Soc. (New Series)* 83(4):745–747.

Krengel U, Sucheston L (1978) On semiamarts, amarts, and processes with finite value. *Probability Banach Spaces* 4:197–266.

Purohit M, Gollapudi S, Raghavan M (2019) Hiring under uncertainty. Chaudhuri K, Salakhutdinov R, eds. *Proc. Internat. Conf. Machine Learn.* (PMLR, New York), 5181–5189.

Samuel-Cahn E (1984) Comparison of threshold stop rules and maximum for independent nonnegative random variables. *Annals Probability* 12(4):1213–1216.

Segev D, Singla S (2021) Efficient approximation schemes for stochastic probing and prophet problems. Biró P, ed. *Proc. 22nd ACM Conf. Econom. Comput.* (Association for Computing Machinery, New York), 793–794.

Yan Q (2011) Mechanism design via correlation gap. Randall D, ed. *Proc. 22nd Annual ACM-SIAM Sympos. Discrete Algorithms* (SIAM, Philadelphia), 710–719.

Boris Epstein is a doctoral candidate in the Decision, Risk, and Operations division of the Graduate School of Business at Columbia University. His research focuses on studying theoretical models of sequential decision making under uncertainty, with applications in revenue management and supply chain management.

Will Ma is an associate professor of Decision, Risk, and Operations at the Graduate School of Business in Columbia University. His research focuses on the theory of online algorithms and their applications in revenue management and e-commerce.